

Computability Exam, Moed C 2022 - Solution

September 27, 2022

Question 1:

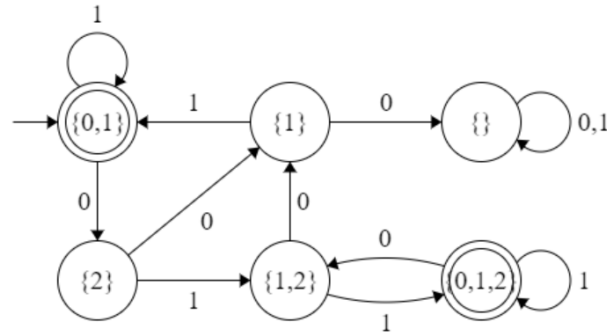


Figure 1: A minimal equivalent DFA

Question 2:

For every $n \geq 1$ consider the word $w_n = 0^n$. Clearly, for every $n \geq m$, we have that $w_n \cdot 1^{2^n} \in L$ and $w_m \cdot 1^{2^n} \notin L$. Thus, 1^{2^n} is a separating tale between w_n and w_m , and the two words are not MN equivalent. That is, L has infinitely many MN equivalence classes, and hence is not regular.

Question 3:

Consider the language $L_{abc} = \{a^n b^n c^n : n \geq 1\}$. We claim that $L_{abc} \in L \setminus CFL$, and hence $L \neq CFL$. It has been proved in class that L_{abc} is not in CFL, and so it is left to prove that $L_{abc} \in L$. We give a logspace algorithm for determining if a word $w \in \{a, b, c\}^*$ is of the form $a^n b^n c^n$ for some $n \geq 1$:

1. Scan the input from left to right. If the word is not of the form $a^+b^+c^+$ then reject. Otherwise, the input is a word $w = a^n b^m c^k$ for $n, m, k \geq 1$, and we write n , m and k in binary on the working tape separated with a special symbol.
2. Denote by x , y and z the three binary strings written on the working tape.
3. While $x > 0$ and $y > 0$ and $z > 0$: Decrease each of the variables x , y and z by 1.
4. If not all variables equal zero then reject, otherwise accept.

Correctness: w is in L_{abc} iff it is of the form $a^n b^m c^k$ for $n = m = k$, and $n = m = k$ iff the three variables x , y and z reach zero at the same iteration of the while loop.

Complexity: Checking if the word is of the form $a^+b^+c^+$ is done only by scanning the input tape (as $a^+b^+c^+$ is a regular condition), converting the number of a 's, b 's and c 's to binary is done in space $O(\log(n) + \log(m) + \log(k)) = O(\log(|w|))$, and then the while loop is done without any additional space.

Question 4:

The language L is in $\overline{\text{RE} \cup \text{coRE}}$. Since $\overline{\text{RE} \cup \text{coRE}}$ is closed under complementation it is sufficient proving that $\overline{L} \in \overline{\text{RE} \cup \text{coRE}}$. We prove that $\text{ALL}_{TM} \leq_m \overline{L}$, and since we saw that $\text{ALL}_{TM} \in \overline{\text{RE} \cup \text{coRE}}$, we conclude by the reduction theorem that $\overline{L} \in \overline{\text{RE} \cup \text{coRE}}$. The reduction to $\overline{L}$ goes as follows. On input x for the reduction, if x is not an encoding of a Turing machine, then the reduction outputs $\langle M_\varepsilon \rangle$ where M_ε is some fixed TM that accepts ε but does not halt on any other input. Otherwise, $x = \langle M \rangle$ for M a TM, and the reduction outputs $\langle M' \rangle$ where M' is the TM that is defined as follows. On any input y for M' , if $y = \varepsilon$ then M' accepts. Otherwise $y = \sigma \cdot y'$ for some $\sigma \in \Sigma$ and $y' \in \Sigma^*$, and then M' simulates M on y' . If M halts and rejects then M' enters an infinite loop (and thus does not halt), and if M halts and accepts then M' accepts.

Correctness

If x is not an encoding of a TM, then $x \notin \text{ALL}_{TM}$ and the reduction outputs $M_\varepsilon \in L$ as required. Otherwise, $x = \langle M \rangle$. First, observe that for any M , the TM M' accepts ε . Thus, it is only left to show that M' halts on every input iff M is universal. Indeed, if $L(M) = \Sigma^*$ then so is $L(M') = \Sigma^*$, and in particular M' halts on every input. Conversely, if $L(M) \neq \Sigma^*$, then there is an input $y' \in \Sigma^*$ such that M either rejects or does not halt on, and in either case M' does not halt on $y = \sigma \cdot y'$ for all $\sigma \in \Sigma$. Thus, M' halts on every input iff M is universal and so $\langle M \rangle \in \text{ALL}_{TM}$ iff $\langle M' \rangle \notin L$.

Computability

Checking if x is an encoding of a TM and if not then returning a constant is computable; Constructing from M a machine that simulates M on input y' is computable (either with a universal machine or by simply changing the structure of M to utilize further functionalities), and so is checking if the input y is empty and otherwise parsing as $y = \sigma \cdot y'$.

Question 5:

Yes. $XYTIME$ is in RE. We describe an NTM N that recognizes $XYTIME$. Note that this is sufficient as any NTM N has an equivalent deterministic TM that recognizes the same language. The NTM N works as follows. On input w , if w is not an encoding of a TM, then N rejects. Otherwise, $w = \langle M \rangle$ for a TM M , and N guesses two words x and y above the alphabet of M . N then simulates M on x and y one after the other while holding counters for the time of each simulation. If both simulations halted and where accepting, then N compares the running times of the two and accepts if they are equal. Otherwise, if at least one simulation was rejecting, or if the simulation times are not equal, then N rejects.

N clearly recognizes $XYTIME$, as $\langle M \rangle \in XYTIME$ iff there exists x and y that can be guesses by N , are both accepted by M and their simulation times are equal.

Question 6:

No. $XYTIME$ is not in $coRE$. Recall that $HALT_{TM}^{\varepsilon} \notin coRE$ where

$$HALT_{TM}^{\varepsilon} = \{\langle M \rangle : M \text{ is a TM that halts on the empty tape}\}$$

We reduce $HALT_{TM}^{\varepsilon}$ to $XYTIME$, and so $XYTIME \notin coRE$. Given an input $\langle M \rangle$, we output $\langle M' \rangle$, where M' is the TM defined as follows.

W.l.o.g we assume that the alphabet of M is not unary (otherwise we add another letter to M 's alphabet). On any input, M' scans the tape from left to right until it reaches the first empty cell. Then it scans all the way back while erasing each cell. Then, after reached to the leftmost cell and finished erasing, M' simulates M on the tape (which is now empty) and accepts if the simulation halted. Clearly M' is universal if M halts on the empty tape, and otherwise the language of M' is empty. Hence, if $\langle M \rangle \notin HALT_{TM}^{\varepsilon}$ then $L(M') = \emptyset$ and in particular $\langle M' \rangle \notin XYTIME$. Moreover, if M halts on the empty tape, then the running time of M' on any word w , is exactly the sum of the time it takes to simulate M on the empty tape, with the time it takes to erase w . The time it takes to erase w only depends on the length of w , and so for any two words x and y , if $|x| = |y|$, then the running times of M' on x and y are the same. More specifically, if $\sigma, \tau \in \Sigma$ are two distinct letters in the alphabet of

M , and M halts on the empty tape, then the words $x = \sigma$ and $y = \tau$ are both accepted by M' and with the same running time. Thus, if $\langle M \rangle \in HALT_{TM}^\varepsilon$ then $\langle M' \rangle \in XYTIME$, and we're done.

Question 7:

The claim is true. First observe that by Savitch's Theorem, for every $i \geq 1$ we have $NSPACE(\log^i(n)) \subseteq SPACE(\log^{2i}(n))$. In addition, for every $i \geq 1$ we have that $SPACE(\log^i(n)) \subseteq NSPACE(\log^i(n))$, as any deterministic machine is also a non-deterministic machine with the same space complexity. Hence,

$$PolyNL \subseteq \bigcup_{i=1}^{\infty} SPACE(\log^{2i}(n)) \subseteq \bigcup_{i=1}^{\infty} SPACE(\log^i(n)) \subseteq \bigcup_{i=1}^{\infty} NSPACE(\log^i(n)) \subseteq PolyNL$$

And we have the following equality,

$$PolyNL = \bigcup_{i=1}^{\infty} SPACE(\log^i(n))$$

Clearly, for any $i \geq 1$, $SPACE(\log^i(n))$ is closed under complementation (as it's a space complexity class of deterministic decider machines), and so $PolyNL$ is closed under complementation. That is,

$$co - PolyNL = \bigcup_{i=1}^{\infty} co - SPACE(\log^i(n)) = \bigcup_{i=1}^{\infty} SPACE(\log^i(n)) = PolyNL$$

Question 8:

The language is NP-Complete. We first observe that a path from s to t that traverses all vertices once but two vertices for which it visits twice, can be verified in polynomial time. We need to check that (1) the witness is a path of length $\ell \in [|V|, |V| + 2]$, and that (2) it begins at s and ends at t , and that (3) each vertex appears at least once in the path. All of those can be done in polynomial time (Note that it is crucial to bound the check of (2) and (3) only to short paths, this is why we first do (1) and check that the path is of the correct length). Hence, $L \in NP$. To see that $LongPath$ is also NP-hard, we reduce in polynomial time $HAMPATH$, which we saw is NP-hard, to L , and hence by the reduction theorem, $LongPath$ is NP-hard.

Reduction from HAMPATH

Let $\langle G, s, t \rangle$ be an encoding of a directed graph $G = \langle V, E \rangle$ and two distinguished vertices s and t . We define the directed graph $G' = \langle V', E' \rangle$ as follows. The vertices of G' are defined $V' = V \cup \{u, v\}$ where u and v are two new vertices

that do not belong to V . The edge set E' of G' is defined by $E' = E \cup \{(s, u), (u, s), (t, v), (v, t)\}$. Note that any path from s to t that traverses also u and v , must do so by traversing the loops $s \rightarrow u \rightarrow s$ and $t \rightarrow v \rightarrow t$, as s and t are the only neighbors of u and v respectively. Hence, any path from s to t that also visits u and v must visit s and t at least twice. It follows that if $p = v_1, v_2, \dots, v_k$ is a witness for $\langle G', s, t \rangle \in \text{LongPath}$, then $(v_1, v_2, v_3) = (s, u, s)$ and $(v_{k-2}, v_{k-1}, v_k) = (t, v, t)$, and hence $p' = v_3, v_4, \dots, v_{k-2}$ is a path from s to t that visits all vertices of G exactly once, and by construction of G' the path p' is also a path in G . Hence, $\langle G', s, t \rangle \in \text{LongPath}$ implies that G has a Hamiltonian path.

Conversely, if $s, v_1, v_2, \dots, v_k, t$ is a Hamiltonian path in G , then the path $s, u, s, v_1, \dots, v_k, t, v, t$ is a witness that $\langle G', s, t \rangle \in \text{LongPath}$.

Overall, $\langle G, s, t \rangle \in \text{HAMPATH}$ iff $\langle G', s, t \rangle \in \text{LongPath}$.

The reduction is computable in polynomial time - we need to add two new vertices, and the rule for adding the edges can be checked in polynomial time (4 edges in between s, t, u and v).

Question 9:

The language is in NL. We describe a non-deterministic logspace algorithm for the complement language, which is sufficient as $\text{NL} = \text{coNL}$: On input $G = \langle V, E \rangle$, for every edge $e \in E$ and every two distinct vertices s and t , guess a path of length at most $|V|$ from s to t that does not traverse e . If such path found for all e, s and t , then accept. Otherwise reject.

Correctness: Clearly $\langle G \rangle \in \text{DISCONN}$ iff there is an edge e and two vertices s and t such that there is no path from s to t that does not traverse e . That is, since the algorithm halts on all inputs, it holds that $\langle G \rangle \in \text{DISCONN}$ iff G is rejected.

Space Complexity: Iterating over all e, s and t requires holding three variables (one for the edge e and two for the vertices s and t) that need space $2 \log(|V|) + \log(|E|)$. Guessing a path of length at most $|V|$ that does not traverse e , requires $\log |V|$ space for the counter of the length of the path, and another $2 \log |V|$ space for the last vertex v and current vertex u in the path, so that we could compare (u, v) with e . Overall the algorithm uses only log space.